

* Properties of Regular languages

- Closure properties of regular languages

If certain languages are regular and L is formed from them by certain operation then L is also regular. These properties are called closure properties of regular languages. Such languages represent the same class of regular which is closed under certain specific operations.

The closure properties of regular languages are given below.

1. Union of two regular languages is Regular.
2. The intersection of two regular language is regular.
3. Complement of regular language is regular.
4. The difference of two regular language is regular.
5. The reversal of regular language is regular.
6. Closure operation on regular language is Regular.
7. Concatenation of regular language is regular.
8. A homomorphism of regular language is regular.
9. The inverse homomorphism of regular language is regular.



* Decision properties of regular languages.
following is the set of questions for determining the decision properties of regular language.

1. Is a particular string N belongs to some language L ?

Let L be a language which can be represent by DFA. If string w is chosen for simulating this DFA processing and then if w reaches to an accept state, starting from the start state.

2. Is the language empty?

The regular language is modeled by finite automata to decide whether it is empty or not. if there exist some path from start state to final state for deriving such languages then the given language is not empty.

3. Do the two description of a languages represent the same language.

Any regular language can be representative by NFA, DFA, or regular expression.

Thus representation can be interconverted. this property is called equivalence of

corresponding two models.

* Pumping Lemma for Regular language

It is used to prove that, language is not regular.

* every finite language is regular.

* If we are able to design a FA for any language then language is regular.

* By Using Pumping Lemma:

If we are able to find some pattern in language then may be that language is regular but if we don't get any pattern then definitely language is not regular.

- Pumping lemma states that,

If L is regular set, then there exist a constant n , such that for every word z in L and $|z| \geq n$, we can write $z = uvw$ in such a way that $|uv| \leq n$, $|v| \geq 1$, and for all $i \geq 0$, $uv^i w$ is in L .

Example -

Prove that $L = \{0^i 2^j / i \geq 1\}$ is non-regular.

Proof -

Let n be a constant of pumping lemma.
select $z = 0^{n^2}$ this ensure that z is in L , $|z| \geq n$.
if we write $z = uvw$ then the possible chances
of satisfying the conditions.

$1 \leq |v| \leq n$ & $|uv| \leq n$ are
 $v = 0^p$; where $1 \leq p \leq n$ i.e. v can be chosen to
be a string of minimum one zero & maximum
of n 0's.

for this choice of v , $|uv^2w|$ (i.e. the length
of uv^2w) will be between n^2+1 & n^2+n
since $n^2+1 > n^2$ & $n^2+n < (n+1)^2$. we
conclude that length of uv^2w is not
a perfect square of any integer,
hence uv^2w can not belong to L .

$\therefore$ we get contradiction to pumping lemma.
hence proving that the given language
is not regular.

* Applications of Regular Expression.

Regular Expression in Unix

There are various unix notation's used for regular expression. these notation's have many additional capabilities & feature. this notations, have ability to name and refer previous strings that have matched pattern. this ability helps in recognising non regular language.

e.g. symbol $\cdot$ stands for single character. sequence $[1, 2, 3, \dots, 10]$ means $1+2+3+\dots+10$. (n) means n copies of

Lexical Analyzer's

Lexical analyzer's used in compiler's. that's of lexical analyze is to scan out the input program & separate out the token's.

Example - Identifier is a category of token in the source language & it can be identified by regular expression.

finding patterns in text.

Regular expressions are useful notations. used for finding the patterns from given text. a large class of pattern can be identified by



regular expression.

* Question Bank

- ① Discuss Application of FA & regular expression. [4]
- ② Using pumping lemma, prove that $\{0^i 2^0 / i \text{ is an integer, } i \geq 1\}$ is not regular. [6]
- ③ prove formula [4]
$$(x^* s^*)^* = (x + s)^*$$
$$(a b)^* \neq a^* b^*$$
- ④ Construct a DFA with reduced state equivalent to $10 + (0 + 11)0^* 1$ [6]
- ⑤ Define Pumping lemma and apply it to prove following [6]
$$L = \{0^m 1^n 0^{m+n} / m \geq 1 \text{ \& } n \geq 1\}$$
 is not regular.

Unit-III

DATE: / /
PAGE NO.:

* Context-free Grammars *

* Chomsky Hierarchy

Types of Grammars.

There are four types of Grammar, differing in their power's.

- 1] type (0) called as unrestricted, phrase, structure
- 2] type (1) context sensitive grammar.
- 3] type (2) context free grammar
- 4] type (3) called as regular grammar.

1] type (0) or Unrestricted Grammar -

A type (0) grammar is the one whose every production rule is one of the form: $\alpha \rightarrow \beta$ where α & β are in $(V \cup T)^*$. $\alpha \neq \epsilon$ they can be string of terminals or non terminals & $\alpha \neq \epsilon$.

2] Type (1) or context-sensitive grammar

A type (1) grammar is the one, whose every production rule is of the form: $\alpha \rightarrow \beta$ where α & β are in $(V \cup T)^*$ & $\alpha \neq \epsilon$, and $|\beta| \geq |\alpha|$.

3] Type (2) or context-free Grammar

A type (2) grammar is the one, whose every production rule is of the form $A \rightarrow \alpha$.



where α is in $(VUT)^*$

- 1) Type (3) or Regular grammar -
Production Rule is not of the form $A \rightarrow aB$ or $A \rightarrow a$, where A, B are in V , and a in T .

1) "Every Regular language is context free language but vice versa is not true".

2) Every context free language not containing epsilon (ϵ) is a context sensitive language but vice versa is not true.

3) Every context sensitive language is recursively enumerable language. vice versa is not true.

* Context-Free Grammar

A CFG is a notation that can be used to specify language, and the language specified by CFG is called as context free language. A CFG consists of terminals as non-terminals start symbol & production. The terminals are nothing but the tokens of language used to form constructs of the language nonterminals are the variables that denote the set of strings.

E.g. - S and E are the non-terminals denoting set of those strings which are statements and expressions respectively in a typical programming language.

Non terminals classifiers. The set of strings that are used to define the language generated by grammar.

Production in a grammar specifies in a way in which terminals and set of strings, defined by non terminals can be combined to form set of strings defined by particular non-terminal start symbol is the starting point of grammar from which of grammar can be read.

∴ Therefore a context free grammar is a four tuple denoted as.

$$G = (V, T, P, S)$$

where, V - finite set of symbols Non terminals

T - finite set of terminals.

P - set of production's

S - is a member of V called as start symbol.

for. eg

$$G = (\{s\}, \{a, b\}, P, s) \text{ when } P \text{ contains}$$

$$\left\{ \begin{array}{l} s \rightarrow asa \\ s \rightarrow bsb \\ s \rightarrow \epsilon \end{array} \right\}$$

* Derivation

It refers to replacement of an instance of a non-terminal in a given string by the right hand side of the production rule, whose left hand side of the production rule, whose replaced. it produces a new string from a given grammar.

∴ derivation can be used repeatedly to obtain a new string from a given string. If string obtained as a result of derivation contains only terminal symbols then no further derivations are possible.

eg. 1. Consider a grammar.

$$G = (\{s\}, \{a, b\}, P, s) \text{ where } P \text{ contains}$$
$$\left\{ \begin{array}{l} s \rightarrow a / bs \\ s \rightarrow a / b \end{array} \right\}$$

eg. 2 Consider a grammar.

$$G = (\{s\}, \{a, b\}, P, s) \text{ where } P \text{ contain}$$
$$\left\{ \begin{array}{l} s \rightarrow asa \\ s \rightarrow bsb \\ s \rightarrow \epsilon \end{array} \right\}$$



Derive a string aabbba. from above grammar

Solⁿ $\rightarrow$ $S \rightarrow aua$
 $S \rightarrow aasaa$ [$S \rightarrow aua$]
 $S \rightarrow aabsbaa$ [$S \rightarrow bsb$]
 $S \rightarrow aabbbaa$ [$S \rightarrow \epsilon$]

$\therefore$ hence aabbbaa is derived from above grammar.

* Standard Notation -

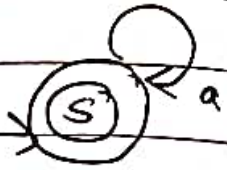
1. The capital letter's towards the start of alphabate of english language are used to denote Nonterminals.
e.g. A, B, C, D etc.
2. Lower case letter's are used to denote terminals.
e.g. a, b, c, d etc....
3. S is used to denote start symbol.

* Language of Grammar -

(The language of grammar can be define by creating the productions rules for given condition language generated by context free grammar is called as context free language.

Example - Construct the CFG for language having any number of a's. over set $\Sigma = \{a\}$.

Soln - We know the regular expression for above mention language is a^* FA is



Let us build production rule for same.

$$\begin{cases} s \rightarrow as \\ s \rightarrow \epsilon \end{cases}$$

$$\therefore G = (\{s\}, \{a\}, P, s)$$

if we want to derive a string aaa the

$$\begin{aligned} s &\rightarrow as \\ &\rightarrow aas && (s \rightarrow as) \\ &\rightarrow aaas && (s \rightarrow as) \\ &\rightarrow aaa && (s \rightarrow \epsilon) \end{aligned}$$

Example Try to recognize the language L for given CFG.

$$G = (\{s\}, \{a, b\}, P, \{s\})$$

$$\text{where } P = \begin{cases} s \rightarrow aab \\ s \rightarrow ab \end{cases}$$

Soln - Given grammar's production rule is $s \rightarrow aab / ab$ if this rule can be effectively applied

$s \rightarrow asb$
 $\rightarrow aasbb \quad (s \rightarrow asb)$
 $\rightarrow aaabbbb \quad (s \rightarrow ab)$

Thus the language is, any number of a's first then equal number of b's following it

$$L: L = a^n b^n \text{ where } n \geq 1$$

* There are two methods of derivation.

- 1] Leftmost derivation
- 2] Rightmost derivation

1] Leftmost derivation -

It is a derivation in which the left most nonterminal is replaced first from the sentential form.

2] Rightmost derivation -

It is a derivation in which the right most non-terminal is replaced first from the sentential form.

Example - Derive the string 1000111 for left & Rightmost derivation using CRG.

$G = (V, T, P, S)$ where

$P = \{ S \rightarrow T^0 T^1 \}$

$T \rightarrow 0T \mid 1T \mid \epsilon$



→ Given Grammar production rule is

$$S \rightarrow T O O T$$
$$T \rightarrow O T \mid I T \mid \epsilon$$

Leftmost Derivation.

1000111

$$\begin{aligned} S &\rightarrow T O O T \\ &\rightarrow I T O O T && [T \rightarrow I T] \\ &\rightarrow I O T O O T && [T \rightarrow O T] \\ &\rightarrow I O O O I T && [T \rightarrow \epsilon] \\ &\rightarrow I O O O I I T && \\ &\rightarrow I O O O I I I T && [T \rightarrow I T] \\ &\rightarrow I O O O I I I && [T \rightarrow \epsilon] \end{aligned}$$

Rightmost Derivation. for 1000111

$$\begin{aligned} S &\rightarrow T O O T \\ &\rightarrow T O O I T && \\ &\rightarrow T O O I I T && [T \rightarrow I T] \\ &\rightarrow T O O I I I T && \\ &\rightarrow T O O I I I && [T \rightarrow \epsilon] \\ &\rightarrow I T O O I I I && \\ &\rightarrow I I O T O O I I I && [T \rightarrow O T] \\ &\rightarrow I O O O I I I && [T \rightarrow \epsilon] \end{aligned}$$

* Parse Tree -

Parse tree is a graphical representation for the derivation of the given production rule for a given CFG. It is a simple way to show how the derivation can be done.

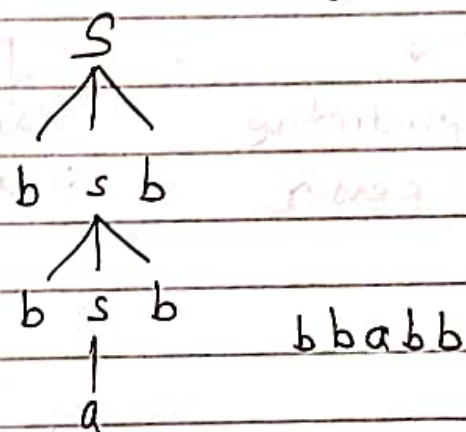
* Properties of parse tree

- 1] Root node is always a node containing start symbol.
- 2] Derivation is read from left to right.
- 3] Left leaf nodes are always a terminal.
- 4] Interior nodes are always non-terminal nodes.

example

$s \rightarrow bsb \mid a \mid b$

generate $bbabbb$ using parse tree



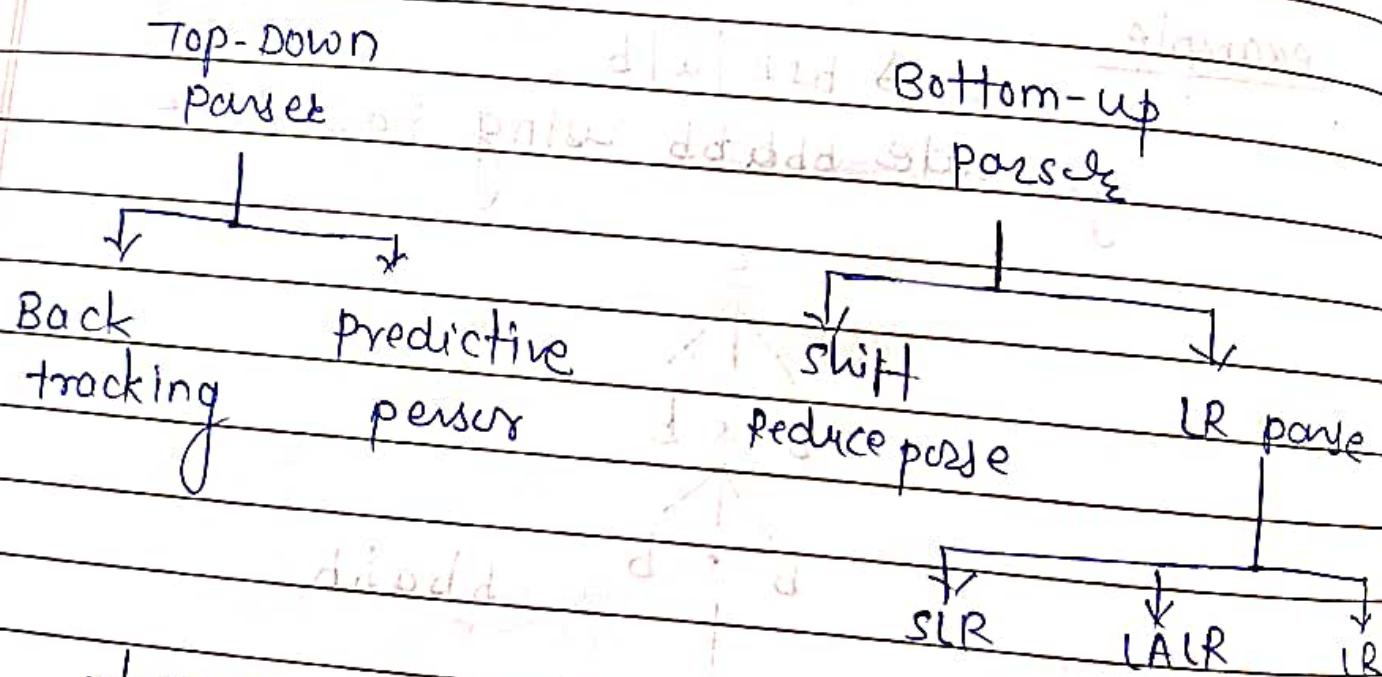
* Application of Context-free Grammar (CFG)

when any high-level program like a or pascal is computed, the compiler checks the syntax of every programming statement by constructing syntax tree. It is necessary to write CFG for each statement, in the programme.

1. Parsing Techniques -

Context-free grammar is used for parsing the programming constructs and finding the syntactical errors from some program. There are two types of parser used by compiler: top-down parser & bottom-up parser.

Parser



when parse tree is constructed from root and expanded to leaves then it is called as TOP-Down.

when parse tree is constructed from leaves and expanded to root then it is called as bottom up parser.

* Ambiguous CFG.

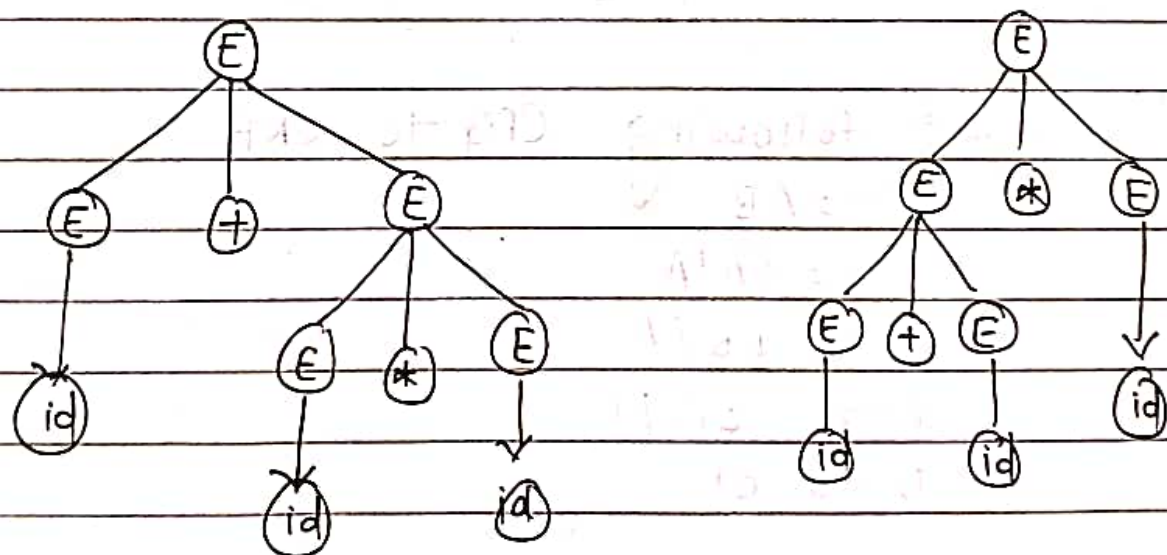
for some $w \in L(G)$, there may exist more than one phrase + parse tree, that means there may exist more than one way of deriving w from G , using the production of grammar.

eg

$$E \rightarrow E + E \mid E * E \mid id$$

we find that, for a string $id + id * id$, there exist more than one parse tree.

shown in following fig.



① $id + id * id$

② $id + id * id$

If there exist more than one parse trees for some $w \in L(G)$ then G is said to be an ambiguous grammar.

Question Bank.

Q.1] Write short note on [8]

1] CNF

2] GNF

2] Simplify the grammar [5]

$S \rightarrow a / xb / aya$

$x \rightarrow y / \epsilon$

$y \rightarrow b / x$

3] Explain with example, any two application of context free grammar [4]

4] Convert following CFG to CNF

$S \rightarrow AB$

$A \rightarrow CA / \Lambda$

$B \rightarrow DB / \Lambda$

$C \rightarrow 011 / 1$

$D \rightarrow 01$

5] Define following with appropriate eg. [6]

1] CFG

2] Derivation Graph.

Unit-IV PUSH DOWN AUTOMATA

DATE: / /

PAGE NO.:

* Basic definition of PDA (Push down Automata).

- Def-: Push Down Automata is an extension of finite automata with control of both an input tape and a stack.

There are two natural ways of the defining the language accepted by a PDA.

1) The language accepted by a PDA is a set of all inputs for which sequence of moves cause the PDA to empty its stack.

2) The language accepted by PDA is a set of all inputs for which some sequences of moves cause the PDA to enter a final state.

$\therefore$ formally PDA M is a system

$(Q, \Sigma, \delta, q_0, \Gamma, z_0, F)$ where

Q - is finite set of states.

Σ - is finite set of input alphabets

q_0 - is in Q an initial state.

Γ - is an alphabet caused stack alphabets.

z_0 - is particular stack symbol.

$F \subseteq Q$ - set of final state

δ - is mapping form $X (\Sigma \cup \{\epsilon\} \times \Gamma^*)$ to final subset of $Q \times \Gamma^*$

ii) Transition function OR Instantaneous Description

The Configuration of PDA at any given instant of time depends on.

1. present stack of finite control

2. The next input symbol which is not yet spent
3. Symbol appearing on the top of the stack

$\therefore$ We formally define the configuration of a PDA at any given instant using a triple (p, x, α) called as instantaneous description or ID, where p is present state, x is next input symbol and α is contents of stack

if there exist a move $(p, a, z) = \{q, AB\}$ then we say :

$$(p, ay, zB) \mid \xrightarrow{M}^* (q, y, AB)$$

$\therefore$ Language accepted by PDA M is

$L(M) = \{w / w \in \Sigma^*, \text{ and } (q_0, w, z_0) \mid \xrightarrow{M}^* (p, \epsilon, \alpha) \text{ if } M \text{ accepting by empty stack. and } L(M) = \{w / w \in \Sigma^* \text{ and } (q_0, w, w) \mid \xrightarrow{M}^* (p, \epsilon, \alpha) \text{ if } M \text{ accepting by final state.}$

(ii) Interpretation of moves / moves for accepting a string.

A move of $(q, a, z) = \{(p_1, y_1) (p_2, y_2) \dots (p_m, y_m)\}$ states that

"If PDA is in state q , with input symbol a and z , top most symbol of stack, then PDA can enter into any p_i and replace symbol z by string y_i (for any p between 1 and m)

Types of PDA:-

- i) Deterministic PDA (DPDA)
- ii) Non-Deterministic PDA (NPDA)

(i) Deterministic Push Down Automata -

DEF -

Let $M = (Q, \Sigma, \Gamma, q_0, z_0, \delta, f / \emptyset)$ be a push down automata for which M has a choice of more than one move.

In other words,

M is deterministic it is satisfy both the following condition.

1] for any $q \in Q$, $a \in \Sigma \cup \{f\}$ and $x \in \Gamma$ the set $\delta(q, a, x)$ has atmost one element.

2] for any $q \in Q$ and $x \in \Gamma$,
if $\epsilon \in \delta(q, \epsilon, x) = \emptyset$, then $\delta(q, a, x) = \emptyset$
for every $a \in \Sigma$.

(ii) Non-Deterministic PDA -

Def - Let $M = (Q, \Sigma, \Gamma, q_0, \Gamma_0, f / \emptyset)$ be a push down automata for which M has a choice of more than one move.

where

Q - is set of finite set of states.

Σ - is finite set of alphabets.

q_0 - is in Q initial state.

F is in Q , set of final states

Γ stack alphabet

20 stack symbol

δ is mapping given by.

$$Q \times (\epsilon \cup \Sigma \cup \Gamma) \times \Gamma^* \rightarrow \{Q \times \Gamma^*\}$$

M can be non-deterministic if it satisfy the condition that for any state q in Q and input alphabet a in $\Sigma \cup \{F\}$, and any stack symbol x in Γ^* the set $\delta(q, ax)$ has more than one element.
and for any

$$\delta(q, \epsilon, x) = \emptyset \text{ then } \delta(q, a, x) = \emptyset$$

Note Instantaneous description and moves for accepting string of DPDA and NDPA are same as that of PDA.

$$Q = (x, p, p) \text{ then } Q = (x, \epsilon, p) \text{ then } Q = (x, \epsilon, p)$$

- what is the difference between finite automata and push down Automata.

finite Automata

Push Down Automata

① FA is simple idealized machine used to recognize patterns within input taken from some character set.

② FA does not have capability to store long sequence of alphabet.

③ FA can be constructed for type (3) grammar.

④ Acceptance in FA of any string is at final state.

⑤ FA can be constructed for regular language.

① PDA is a type of automata that employs a stack.

② PDA has stack to store the input alphabet.

③ PDA can be constructed for type (2) grammar.

④ Input alphabet are accepted by reading
① Empty stack
② final state

⑤ PDA is constructed by context free language

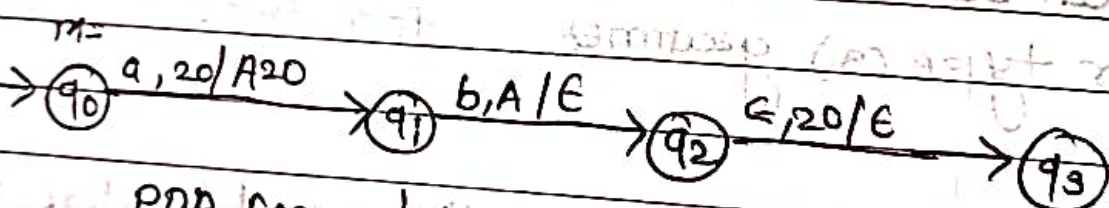
5.2 Equivalence of Acceptance by final state and empty stack.

Theorem 1:

for every PDA accepting empty stack there exist an equivalent PDA accepting by final state. where the equivalence is different defined in terms of language acceptance.

Proof -:

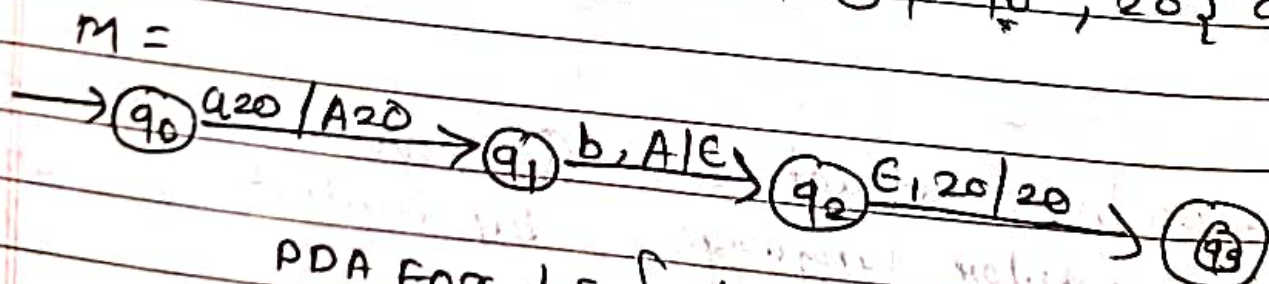
$M = (Q, \Sigma, \delta, q_0, f, \Gamma, z_0, \emptyset)$ be a PDA for $L = \{a, b\}$, accepting by empty stack as follows:



PDA for string 'ab' by empty stack

then let us construct M_1 using M as follows

$$M_1 = \{Q \cup \{q_0', q_f\}, \Sigma, \delta', q_0', z_0, \{q_f\}$$



PDA for $L = \{w / w \text{ is string } ab\}$
by final state

constantly M_1 accepts $L(M)$, because M_1 enters from its initial configuration into the initial configuration of M without consuming any input symbol. then it carries out the simulation of M , and when M empties its stack, then M_1 enters into its final state thereby proving for every PDA accepting by empty stack, there exist an equivalent PDA accepting by final state.

Theorem 2:- For every PDA accepting by final state, there exist an equivalent PDA accepting by empty stack.

that means for every PDA accepting by final state there exists a PDA accepting the same language by empty stack.

Proof -:

let $M = (Q, \Sigma, \Gamma, q_0, f, \delta, z_0)$ be a PDA accepting by empty stack.

let us construct M_1 as follows:

$$M_1 = (Q \cup \{q_0', q_c\}, \Sigma, \Gamma \cup \{x_0\}, \delta', q_0', x_0, \delta')$$

where

$$\delta'(q_0', \epsilon, z_0) = \{q_0, z_0 x_0\}$$

$$\delta'(q, a, z) = \delta(q, a, z) \text{ for all } q \in Q$$

$a \in \Sigma \cup \{\epsilon\}$ and $z \in \Gamma$

for all $q \in F$ and $z \in \Gamma \cup \{x_0\}$,

$$\delta'(q, \epsilon, z) \text{ contains } (q_c, \epsilon)$$

for all x in Σ^* $\{x_0\}$, M contains (q, ϵ)
 contains M accepts $L(M)$, because M
 first enters from its initial configuration
 into the initial configuration of M ,
 without consuming any input symbol.
 then it carries out the simulation of M
 and when M enters into its final state
 then M empties its stack. There by
 proving that for every PDA accepting by
 final state, there exist an equivalent PDA
 accepting by empty stack.

Que 7] What are two different ways to define acceptability?

Soln. - There are two different ways to define PDA acceptability.

- 1] Final state Acceptability
- 2] Empty state Acceptability

1] Final state Acceptability - In final state acceptability a PDA accepts a string when, after reading the entire string, the PDA is in a final state. From the starting state, we can make moves that end up in a final state with any stack values. The stack values are irrelevant as long as we end up in a final state.

for a PDA $(Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$, the language accepted by the set of final state F is.

$$L(PDA) = \{w \mid (q_0, w, z_0) \vdash^* (q, \epsilon, x), q \in F\}$$

for any input stack x .

2) Empty stack Acceptability -

Here, PDA accepts a string when, after reading the entire string, the PDA has an emptied its stack.

For a PDA $(Q, \Sigma, \Gamma, q_0, T, z_0, \emptyset)$ the language accepted by an empty stack is -

$$L(PDA) = \{w \mid (q_0, w, z_0) \vdash^* (q_1, \epsilon, \epsilon), q_1 \in Q\}$$

- Examples on PDA/DPDA

Q1) Construct a PDA or DPDA that accepts $L = \{a^n b^n \mid n \geq 1\}$ by empty stack.

Solⁿ - PDA moves for Accepting $L = \{a^n b^n \mid n \geq 1\}$ is

$$\delta(q_0, a, z_0) = (q_0, az_0)$$

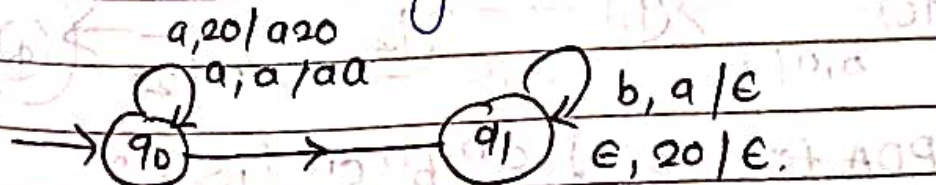
$$\delta(q_0, a, a) = (q_0, aa)$$

$$\delta(q_0, b, a) = (q_1, \epsilon)$$

$$\delta(q_1, b, a) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, z_0) = (q_1, \epsilon)$$

Transition diagram





$\therefore$ PDA is,
 $M = \langle \{q_0, q_1\}, \{a, b\}, \delta, q_0, \{q, 20\}, 20, \delta \rangle$
where δ is given above.

Q.2] Construct PDA or DPDA that accepts
 $L = \{a^n b^n c^n / n \geq 1\}$ by final state.

Solution - [Logic]

1st $\rightarrow$ push for a, skip b, pop for c OR
2nd $\rightarrow$ push for a, pop for b, skip c

PDA moves for accepting $L = \{a^n b^n c^n / n \geq 1\}$
[* push for a, skip b, pop for c]

$$\delta(q_0, a, 20) = (q_0, a20)$$

$$\delta(q_0, a, a) = (q_0, aa)$$

$$\delta(q_0, b, a) = (q_1, a)$$

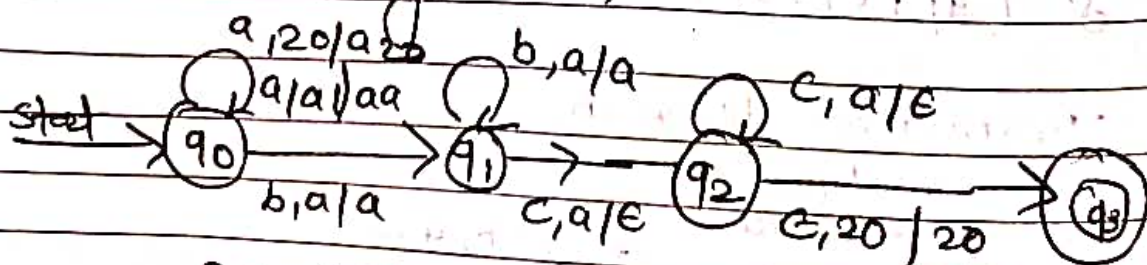
$$\delta(q_1, b, a) = (q_1, a)$$

$$\delta(q_1, c, a) = (q_2, \epsilon)$$

$$\delta(q_2, c, a) = (q_2, \epsilon)$$

$$\delta(q_2, \epsilon, 20) = (q_3, 20)$$

Transition diagram is,



PDA for $L = \{a^n b^n c^n / n \geq 1\}$

∴ PDA is

$$M = (\{q_0, q_1, q_2, q_3\}, \{a, b\}, \delta, q_0, \{q_3\}, \{a, z_0\}, \{z_0\})$$

where δ is given above

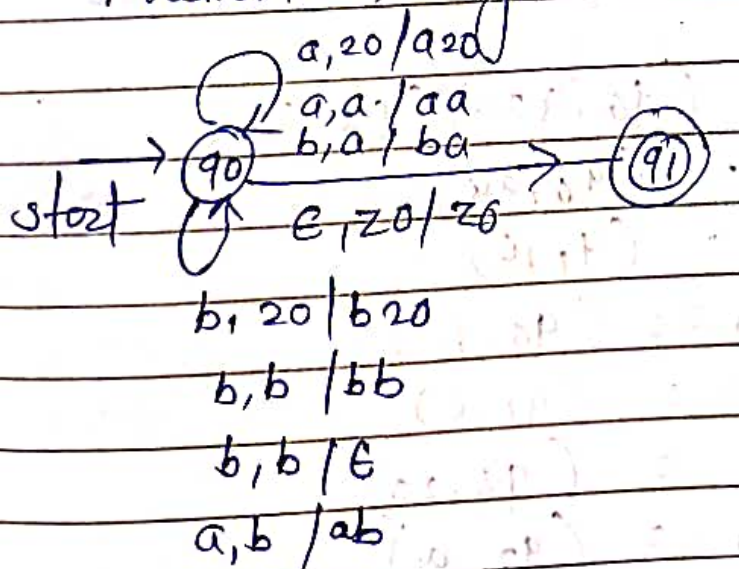
* Examples ON Non-Deterministic PDA - NPDA

Ex. 1 Construct PDA/NPDA for accepting the language $L = \{ww^R \mid w \in \{a+b\}^*\}$ by final state.

Solⁿ - The NPDA moves for accepting $L = \{ww^R \mid w \in \{a+b\}^*\}$

$$\begin{aligned} \delta(q_0, a, z_0) &= (q_0, az_0), \delta(q_0, b, z_0) = (q_0, bz_0), \\ \delta(q_0, a, a) &= (q_0, aa), \delta(q_0, b, b) = (q_0, bb), \\ \delta(q_0, b, a) &= (q_0, ba), \delta(q_0, a, b) = (q_0, ab), \\ \delta(q_0, \epsilon, z_0) &= (q_1, z_0) \end{aligned}$$

Transition diagram is



∴ NPDA is

$M = (\{q_0, q_1\}, \{a, b\}, \delta, q_0, \{q_1\}, \{a, b\}, \epsilon)$
where δ is given above

Let us simulate this PDA for input string
'abba'

$\delta(q_0, abba, 20) \vdash (q_0, bba, 20)$

$\vdash (q_0, ba, 20)$

$\vdash (q_0, a, 20)$

$\vdash (q_0, \epsilon, 20)$

$\vdash (q_1, 20)$

Accepted by final state

Q.2] Construct a NPDA/PDA for accepting language
 $L = \{a^i, b^j, c^k \mid i \neq j \text{ or } j \neq k\}$
final state $i, j, k \geq 1$

→ The NPDA moves for accepting
 $L = \{a^i, b^j, c^k \mid i \neq j \text{ or } j \neq k\}$ is

$\delta(q_0, a, 20) = (q_0, a, 20)$

$\delta(q_0, a, a) = (q_0, aa)$

$\delta(q_0, b, a) = (q_1, \epsilon)$

$\delta(q_0, b, 20) = (q_0, b, 20)$

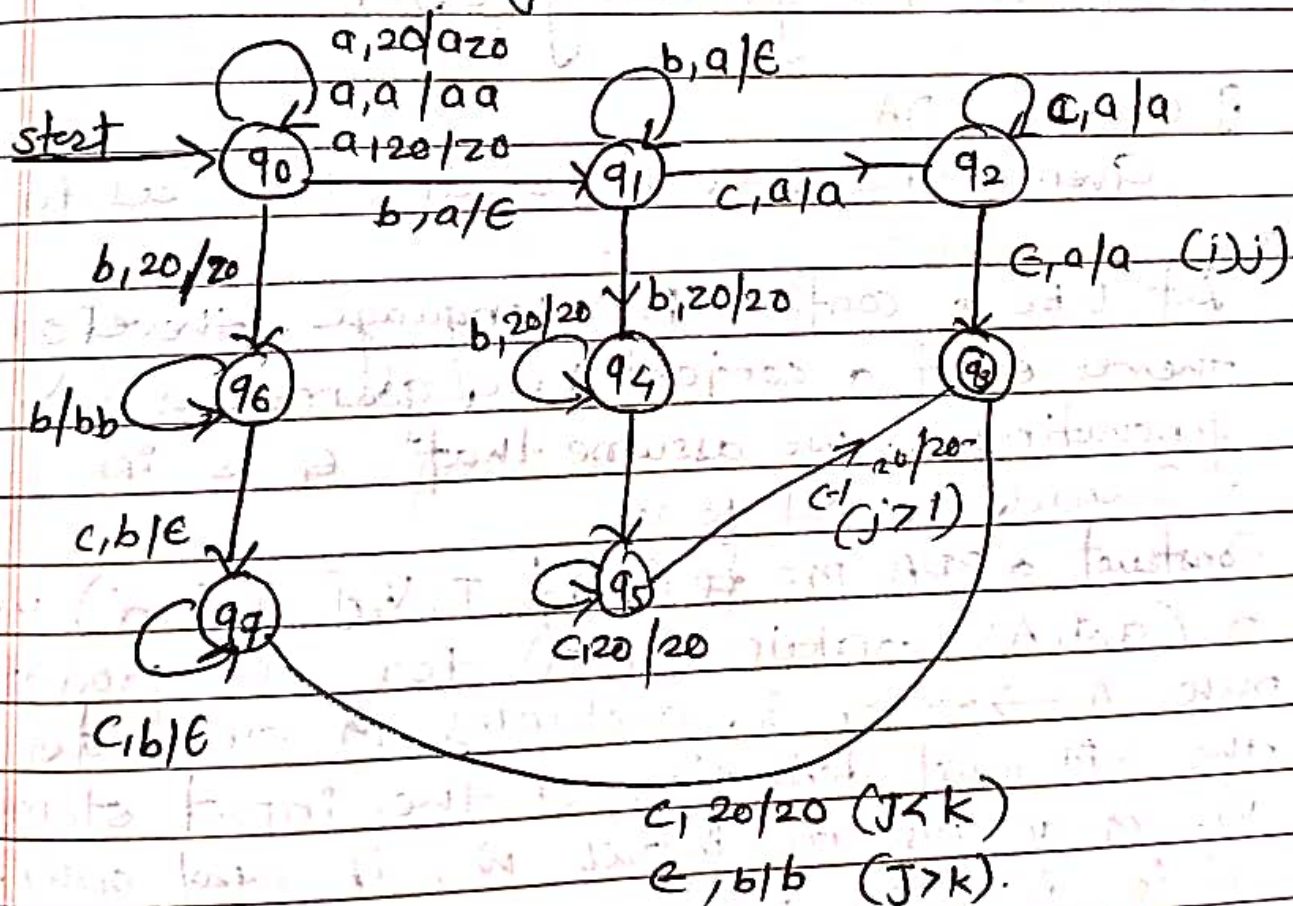
$\delta(q_1, b, a) = (q_0, \epsilon)$

$\delta(q_1, b, 20) = (q_1, 20)$

$\delta(q_1, c, a) = (q_2, a)$

$$\begin{aligned} \delta(q_2, c, a) &= (q_2, a) \\ \delta(q_2, \epsilon, a) &= (q_3, a) \\ \delta(q_4, b, 20) &= (q_4, 20) \\ \delta(q_4, c, 20) &= (q_8, 20) \\ \delta(q_5, c, 20) &= (q_5, 20) \\ \delta(q_5, \epsilon, 20) &= (q_5, 20) \\ \delta(q_6, b, b) &= (q_6, \epsilon) \\ \delta(q_6, c, b) &= (q_7, \epsilon) \\ \delta(q_7, c, b) &= (q_7, \epsilon) \\ \delta(q_7, \epsilon, b) &= (q_3, b) \\ \delta(q_7, c, 20) &= (q_3, 20) \end{aligned}$$

Transition diagram -



Let all simulate string $aabc$ ($i > j$)

$$\begin{aligned} \delta(q_0, aabc, 0) &\vdash \delta(q_0, abc, a20) \\ &\vdash \delta(q_0, bc, aa20) \\ &\vdash \delta(q_1, c, a, 20) \\ &\vdash \delta(q_2, \epsilon, a20) \\ &\vdash q_3 \text{ final state} \\ &(i > j) \end{aligned}$$

* Equivalence of PDA & CFG

For every CFG, G . There exist a PDA, M accepting $L(G)$ and for every PDA M , There exist C.F.G a generating $L(M)$

1] CFG TO PDA

Given a CFG we can obtain PDA as follows:

Let L be a context free language, therefore there exist a context free grammar $G = (V, T, P, S)$ generating L . We assume that G is in Greibach normal form.

Construct a PDA $M = (\{q\}, T, V, \delta, q, S, \epsilon)$ where $\delta(a, a, A)$ contain (q, α) for each production rule $A \rightarrow \alpha$ of G . certainly M simulates the left most derivation of the input strings w . M accepts its input w , if and only if w is in $L(G)$. hence M is a PDA accepting $L(G) = L$.

Step 1 - Check given grammar is in GNF or not
if it is GNF convert to PDA.

Step 2 - If G is not in GNF, convert grammar into GNF and then its equivalent PDA.

Example -

Construct a PDA equivalent to following grammar.

$S \rightarrow aAA$

$A \rightarrow as | bs | a$

Solution - Step 1 - Given grammar is already in GNF form

Step 2 - The PDA equivalent to the given grammar

$$M = (\{q_0\}, \{a, b\}, \{s, A\}, \delta, q_0, s, \emptyset)$$

where,

$$\delta(q_0, a, s) = \{(q_0, AA) \text{ because of } s \rightarrow aAA\}$$

$$\delta(q_0, a, A) = (q_0, s) (q_0, \epsilon) \text{ because of } A \rightarrow as/a$$

$$\delta(q_0, b, A) = (q_0, s) \text{ because of } A \rightarrow bs$$

This is equivalent PDA

* Note -

Grammar generated $(A, \{a, b\}, \{s, A\}, \delta, q_0, s, \emptyset) = M$

$$1) s \rightarrow aAB \text{ because } (A, \{a, b\}, \{s, A\}, \delta, q_0, s, \emptyset) = M$$

$$2) A \rightarrow bA \text{ because } (A, \{a, b\}, \{s, A\}, \delta, q_0, s, \emptyset) = M$$

$$3) B \rightarrow \epsilon \text{ because } (A, \{a, b\}, \{s, A\}, \delta, q_0, s, \emptyset) = M$$

$$4) \epsilon \rightarrow aDB \text{ because } (A, \{a, b\}, \{s, A\}, \delta, q_0, s, \emptyset) = M$$

Example 2 - Construct an equivalent PDA for following grammar

$$S \rightarrow CA$$

$$A \rightarrow a$$

$$C \rightarrow aB/b$$

Solution - Step 1 - Convert given grammar to GNF
in above grammar $S \rightarrow CA$ is not in GNF

1) Remove ϵ production:

There is no ϵ production in given grammar.

2) Remove unit production: no unit production in above grammar.

3) Remove useless symbols: no useless symbols.

Now $S \rightarrow CA$ can be converted to GNF by substituting $C \rightarrow aC/b$

$$\therefore S \rightarrow aCA/bA$$

$$A \rightarrow a$$

$$C \rightarrow aC/b$$

Step 2 - Equivalent PDA is

$$M = (\{q_0\}, \{a, b\}, \{S, A, C\}, \delta, q_0, S, \emptyset)$$

where δ is

$$\delta(q_0, a, S) = (q_0, CA) \text{ because } S \rightarrow aCA$$

$$\delta(q_0, b, S) = (q_0, A) \text{ because } S \rightarrow bA$$

$$\delta(q_0, a, A) = (q_0, \epsilon) \text{ because } A \rightarrow a$$

$$\delta(q_0, a, C) = (q_0, C) \text{ because } C \rightarrow aC$$

$$\delta(q_0, b, C) = (q_0, \epsilon) \text{ because } C \rightarrow b$$

* PDA to CFG

Given a PDA accepting a language L , we can obtain a CFG generating L as follows:

Let M be a PDA then there exist a CFG $G = (V, \Sigma, P)$ where V is the set of objects of the form $[q, A, p]$ where p, q are in Q and A is in Γ .

* Note:

PDA

CFG

1) if $(q_0, \epsilon, z_0) = (q_0, \epsilon)$ then $[q_0, z_0, q_0] \rightarrow \epsilon$

2) if $(q_1, 1, x) = \{(q_1, \epsilon)\}$ then $[q_1, x, q_1] \rightarrow 1$

if

3) $(q_1, 0, z_0) = (q_0, z_0)$ then

$[q_1, z_0, q_0] \rightarrow 0 \cdot [q_0, z_0, q_0]$

$[q_1, z_0, q_1] \rightarrow 0 \cdot [q_0, z_0, q_1]$

if $\delta(q_0, 1, z_0) = \{(q_1, x, z_0)\}$

then

$[q_0, z_0, q_0] \rightarrow 1 \cdot [q_1, x, q_0] \cdot [q_0, z_0, q_0]$

$[q_0, z_0, q_0] \rightarrow 1 \cdot [q_1, x, q_1] \cdot [q_1, z_0, q_0]$

$[q_0, z_0, q_1] \rightarrow 1 \cdot [q_1, x, q_0] \cdot [q_0, z_0, q_1]$

$[q_0, z_0, q_1] \rightarrow 1 \cdot [q_2, x, q_1] \cdot [q_1, z_0, q_1]$

2] for the PDA $M = (\{q_0, q_1\}, \{0, 1\}, \{0, 1, 20\}, \delta, q_0, 20)$ where δ is given below.

$$\delta(q_0, \epsilon, 20) = (q_1, \epsilon)$$

$$\delta(q_0, 0, 20) = (q_0, 020)$$

$$\delta(q_0, 0, 0) = (q_0, 00)$$

$$\delta(q_0, 1, 1) = (q_0, 0)$$

$$\delta(q_0, 0, 1) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, 20) = (q_1, \epsilon)$$

→ The production of equivalent CFG & GFG are obtain as follows:

$$S \rightarrow [q_0, 20, q_0]$$

$$S \rightarrow [q_0, 20, q_1]$$

Because there are two states q_0, q_1 , therefore the grammar should generate all those strings that allow PDA to M to read 20.

for a move $\delta(q_0, 0, 20) = (q_0, 020)$ we get
 $[q_0, 20, q_0] \rightarrow 0$

for a move $\delta(q_0, \epsilon, 20) = (q_1, \epsilon)$

we get $[q_0, 20, q_1] \rightarrow \epsilon$. — ①

for a move $\delta(q_0, 0, 20) = (q_0, 020)$, we get

$$[q_0, 20, q_0] \rightarrow 0 [q_0, 0, q_0] [q_0, 20, q_0]$$

$$[q_0, 20, q_0] \rightarrow 0 [q_0, 0, q_1] [q_1, 20, q_0]$$

$$[q_0, 20, q_1] \rightarrow 0 [q_0, 0, q_0] [q_0, 20, q_1]$$

$$[q_0, 20, q_1] \rightarrow 0 [q_0, 0, q_1] [q_1, 20, q_1]$$

for a move $\delta(q_0, 0, 0) = (q_0, 00)$, we get

$$[q_0, 0, q_0] \rightarrow 0 [q_0, 0, q_0] [q_0, 0, q_0]$$

$$[q_0, 0, q_0] \rightarrow 0 [q_0, 0, q_1] [q_1, 0, q_0]$$

$$[q_0, 0, q_1] \rightarrow 0 [q_0, 0, q_0] [q_0, 0, q_1]$$

$$[q_0, 0, q_1] \rightarrow 0 [q_0, 0, q_1] [q_1, 0, q_1]$$

for a move $\delta(q_0, 1, 1) = (q_0, 0)$, we get

$$[q_0, 1, q_0] \rightarrow 1 [q_0, 0, q_0]$$

$$[q_0, 1, q_1] \rightarrow 1 [q_0, 0, q_1]$$

for a move $\delta(q_0, 0, 1) = (q_1, \epsilon)$, we get

$$[q_0, 1, q_1] \rightarrow 0 [q_1, \epsilon, q_1]$$

for a move $\delta(q_1, \epsilon, 20) = (q_1, \epsilon)$, we get

$$[q_1, 20, q_1] \rightarrow \epsilon [q_1, \epsilon, q_1]$$

If we rename the object as follows

object	Name
$[q_0, 20, q_0]$	A $[q_1, 0, q_0] \rightarrow I$
$[q_0, 20, q_1]$	B $[q_1, 0, q_1] \rightarrow J$
$[q_0, 0, q_0]$	C
$[q_0, 0, q_1]$	D
$[q_0, 1, q_0]$	E
$[q_0, 1, q_1]$	F
$[q_1, 20, q_1]$	G
$[q_1, 20, q_0]$	H

production can be written as

$G = \{ S \rightarrow A/B$
 $B \rightarrow OCB/ODG/E$
 $A \rightarrow OCA/ODH$
 $C \rightarrow OCC/ODJ$
 $D \rightarrow OCD/ODJ$
 $E \rightarrow IC$
 $F \rightarrow ID/O$
 $H \rightarrow E$

- Parsing & PDA

Suppose that G is a context-free grammar over an alphabet Σ . To parse a string $x \in \Sigma^*$ is often useful. Parsing a statement in a programming language is necessary in order to classify it according to syntax, parsing all regular expression is essentially what allows us to evaluate expressive.

1. TOP DOWN parsing -

When the parser starts constructing the parse tree from the start symbol & then tries to transform the start symbol to the input, it is called as top-down parsing.

* Recursive descent parsing -

It is a common form of top down parsing which uses recursive procedure to

process the input. Recursive descent parsing suffers from backtracking.

* Backtracking:

It means, if one derivation of production fails, the syntax analyse restarts the process using different rules of same production. The technique may process the input strings more than once to determine the right production.

* Predictive parser:

It is a recursive descent parser which has the capability to predict which production is to be used to replace the input string. The predictive parser does not suffer from backtracking.

To accomplish its task, predictive parser uses a look ahead pointer, which points to the next input symbol. To make the parser back-tracking free, the predictive parser puts some constraints to the grammar known as LL(k) parser.

2] Top-Down parsing for DPDA -

For top down parsing, a PDA has following four types of transition,

c) pop the non terminal on the left hand side & the production on the left hand side of the production at the top of the stack and push its right-hand side string.

d) IF the top symbol & the stack matches with the input symbol being read, pop it.

e) push the start symbol 's' in to the stack.

f) IF the input string is fully read & the stack is empty, go the final state F.

Example - Design a top-down parser for the expression $(x+y \cdot z)$.

Solution - IF the PDA is $(Q, \Sigma, S, \delta, q_0, 1, F)$ then the top-down parsing is:

$(x+y \cdot z, 1) \vdash (x+y \cdot z, SI)$

$\vdash (x+y \cdot z, S+2EI)$

$\vdash (x+y \cdot z, x+x1)$

$\vdash (x+y \cdot z, y+x1)$

$\vdash (x+y \cdot z, x+x1)$

$\vdash (x+y \cdot z, +x1)$

$\vdash (y \cdot z, x1)$

$\vdash (y \cdot z, x \cdot y1)$

$\vdash (y \cdot z, y \cdot y1)$



$$T(2, 4)$$

(Question Bank -)

$$T(2, 2)$$

$$T(1, 1)$$

1) Differentiate between FA and DFA

Answer: FA is a finite automaton which accepts the language. DFA is a finite automaton which accepts the language and is a subset of FA.

2) Illustrate the working of shift & reduce parser for id + id * id

Consider following grammar

$$E \rightarrow E + E \mid E * E \mid id$$

$$T \rightarrow T * T \mid id$$

$$F \rightarrow F / F \mid id$$

$$E \rightarrow id$$

3) What are the two different ways to define PDA acceptability?

4) What is PDA? What are the different types of PDA? Give its application.

5) Explain the PDA for $\{a^n b^n\}$.

Answer: PDA is a finite automaton which accepts the language and is a subset of FA.

Question Bank -

- 1] Differentiate between FA and PDA. 4 marks
- 2] Construct PDA that accepts the language generated by $S = (S + S / S * S) / 3$ - 6 marks
- 3] Illustrate the working of shift & reduce parser for $id + id * id$ - 8 marks
Consider following grammar.
$$E \rightarrow E + E / T$$
$$T \rightarrow T * F / F$$
$$F \rightarrow \{E\} / id \quad \text{--- unit 3}$$
- 4] What are the two different ways to define PDA acceptability.
- 5] What is PDA? What are different types of PDA? Give its application.
- 6] Obtain CFG for PDA given below - 9 marks
 $M = \{ \{q_0, q_1\}, \{0, 1\}, \{20, x\}, \delta, q_0, 20, \epsilon \}$
when δ is given as.